



# *Extra* Frontend Cine con React

16/12/25

---

José Manuel Sánchez Rosal

2 DAM

Antonio Gala

14700 Palma del Río (Córdoba)

## Índice

|                                                               |          |
|---------------------------------------------------------------|----------|
| <b>Índice.....</b>                                            | <b>1</b> |
| <b>Objetivos.....</b>                                         | <b>1</b> |
| <b>Fase 1: Adaptación del Backend (Spring Boot).....</b>      | <b>1</b> |
| 1.1 Modificación de Entidades y Servicios.....                | 1        |
| 1.2 Integración con TMDB (The Movie Database).....            | 2        |
| <b>Fase 2: Configuración del Frontend (React + Vite).....</b> | <b>2</b> |
| 2.1 Inicialización del Proyecto.....                          | 2        |
| 2.2 Gestión de Dependencias.....                              | 2        |
| <b>Fase 3: Integración Backend-Frontend (CORS).....</b>       | <b>3</b> |
| <b>Fase 4: Funcionalidades Avanzadas.....</b>                 | <b>3</b> |
| 4.1 Importador Automático.....                                | 3        |
| 4.2 Trailers.....                                             | 3        |
| 4.3 Sistema de Críticas y Bot.....                            | 3        |
| <b>Fase 5: Navegación y Lógica de UI.....</b>                 | <b>4</b> |
| Lógica de Control de Vistas.....                              | 4        |

## Objetivos

Esta documentación detalla el proceso de transformación de una API REST de Spring Boot convencional en una plataforma de streaming tipo Netflix ("Joseph\_Link"), integrando un frontend moderno en React y consumiendo datos externos de TMDB.

## Fase 1: Adaptación del Backend (Spring Boot)

El objetivo inicial fue adaptar la arquitectura existente para soportar imágenes de alta calidad y sincronización externa.

### 1.1 Modificación de Entidades y Servicios

- **Actualización del modelo:** Se modificaron las clases **Pelicula**, **PeliculaDTO**, **PeliculaService** y **PeliculaController**.
- **Nuevos campos:** Se añadieron atributos para **foto vertical** y **póster HD** para mejorar la estética visual en el frontend.

## 1.2 Integración con TMDB (The Movie Database)

- **Servicio TMDB:** Se creó la clase **TMDBService** encargada de recorrer las películas existentes en la base de datos local y solicitar las imágenes correspondientes a la API de TMDB.
- **Sincronización:** Se implementó un método específico para actualizar masivamente la información de las películas en la base de datos.
- **Validación:** Se realizaron pruebas de los endpoints de sincronización utilizando **Postman** para verificar la correcta actualización de los datos.
- **Nota técnica:** Debido a que las sinopsis de TMDB son extensas, fue necesario modificar el archivo **application.yml** y recrear la base de datos para ajustar los límites de caracteres

## Fase 2: Configuración del Frontend (React + Vite)

Creación del entorno de desarrollo para la interfaz de usuario.

### 2.1 Inicialización del Proyecto

1. Verificación de entorno Node.js: `node -v`.
2. Creación del proyecto utilizando **Vite**:
  - Comando: `npm create vite@latest frontend-cine -- --template react`.
  - Configuración: React, JavaScript + SWC.

### 2.2 Gestión de Dependencias

Instalación de librerías clave dentro del directorio del proyecto: **frontend-cine**:

- **Core:** `npm install`
- **Utilidades:** `npm install axios swiper react-router-dom lucide-react` (para peticiones HTTP, carruseles, enrutamiento e iconos).
- **Estilos (Tailwind CSS):**
  - Instalación: `npm install -D tailwindcss postcss autoprefixer`.
  - Inicialización: `npx tailwindcss init -p`

**Solución de errores:** Si falla la creación automática de `tailwind.config.js` y `postcss.config.js`, se deben crear manualmente e instalar el plugin específico: `npm install -D @tailwindcss/postcss`

## Fase 3: Integración Backend-Frontend (CORS)

Conexión entre el servidor Java (puerto 8081) y el cliente React (puerto 5172).

1. **Desbloqueo de Puertos:** Para solucionar los errores de CORS (Cross-Origin Resource Sharing) que bloquean la conexión.
2. **Configuración en Controller:** Se añadió la anotación `@CrossOrigin(origins = "*")` en `PeliculaController` para permitir peticiones desde cualquier origen durante el desarrollo.
3. **Resultado Visual:** Tras limpiar la página de ejemplo en `App.jsx`, se logró la primera conexión exitosa mostrando el boceto de la plataforma bajo el nombre "Joseph\_Link".

## Fase 4: Funcionalidades Avanzadas

Implementación de características dinámicas para enriquecer la experiencia de usuario.

### 4.1 Importador Automático

Se evolucionó el `TmdbService` para no solo actualizar imágenes, sino importar listas completas de películas (paginación de 60 en 60) directamente desde la API externa, automatizando el llenado de contenido.

### 4.2 Trailers

- Se modificó la entidad `Pelicula` y su DTO para incluir el `trailerKey` (string identificador del video).
- **Filtro de Idioma:** Se implementó lógica para priorizar y seleccionar trailers que estén en español.

## 4.3 Sistema de Críticas

- Se añadieron las entidades y relaciones para gestionar críticas (lista de críticas en Películas y usuario en Crítica)
- **Bot Generador:** Se desarrolló un script para generar críticas aleatorias automáticamente para cada película, poblando la sección de opiniones, **pero más tarde se decidió cambiarlo por críticas reales para darle más veracidad a la app.**

## Fase 5: Navegación y Lógica de UI

Se implementó una lógica de navegación fluida en `App.jsx` controlada por la función `changeView`

### Lógica de Control de Vistas

La función `changeView` actúa como el "cerebro" de la navegación, realizando las siguientes acciones al cambiar de sección:

- Actualiza el estado `view`.
- Ejecuta `window.scrollTo` para llevar al usuario al inicio de la página.
- Limpia la barra de búsqueda para evitar conflictos visuales.